

AI 100 講 — 補充單元

Claude Code 原始碼洩露

從 51 萬行程式碼學到的 AI 使用秘訣

2026-03-31 — 全球最頂尖 AI 公司的一次「人為疏失」

按 → 或空白鍵開始

今天的旅程

1 事件始末

一個 .npmignore 漏加的故事

2 引擎 vs 整台車

51 萬行程式碼裡，AI 只佔 5%

3 CLAUDE.md 的秘密

每輪重載、40,000 字元、六條黃金規則

4 多 Agent 與快取

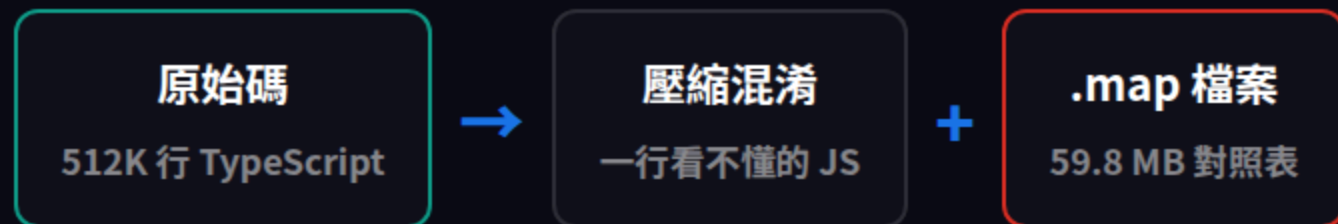
5 個 Agent 的成本 $\approx$ 1 個

5 上下文就是命

你的對話正在被壓縮，你知道嗎？

什麼是 Source Map ?

程式碼發布時會「壓縮混淆」，Source Map 是還原用的對照表



就像買一台車，結果整台車的設計圖、引擎規格、安全系統文件全部放在手套箱裡

Anthropic 忘了把 .map 加入 .npmignore

事件時間軸

1 npm 發布 v2.1.88

Bun bundler 預設產生 source map，沒人注意到

2 安全研究員 Chaofan Shou 發現

59.8 MB 的 .map 檔案就在 npm 套件裡

3 4 小時內全球擴散

GitHub 湧入 41,500+ forks，成為史上最快破紀錄的 repo

4 Anthropic 下架 + DMCA

下架套件、發出 DMCA，封鎖 8,100+ repos

! 這是第二次犯同樣的錯

2025 年第一次發布時就出過一模一樣的事故

為什麼這件事值得關注？



不是洩密醜聞

模型權重、訓練資料都沒有洩露。洩露的是 AI 工具的「操作系統」



是學習機會

全球最好的 AI 工具是怎麼設計的？原始碼直接告訴你



也是提醒

AI 時代最大的風險，不是 AI 太強，是人在基礎操作上的疏忽

關鍵洞察：全球最頂尖的 AI 公司，在最基礎的 build pipeline 上翻車兩次

Part 2

引擎 vs 整台車

51 萬行程式碼裡，AI 只佔 5%

原始碼規模

512K

行程式碼

1,906

檔案

~5%

呼叫 LLM API

~95%

Harness 控制系
統

模型是引擎，Harness 是整台車。

引擎再強，沒有煞車、方向盤和變速箱，你哪也去不了。

Harness 架構圖



同樣的引擎，不同的車

為什麼同樣呼叫 Claude API，產品體驗天差地別？

面向	只有引擎	完整的車
安全	使用者自己負責	23-25 項自動檢查
記憶	每次對話從零開始	三層持久化記憶
上下文	直到爆掉才知道	四階段智慧壓縮
工具	純文字來回	40 工具直接操作系統
並行	只能單線程	多 Agent + 快取共享

Tool System — 你每天在用的

40+

內建工具

50+

Slash 指令

29K

行工具定義程式碼



**Read /
Write /
Edit**

讀寫檔案，比
cat 或 sed 更安
全



**Grep /
Glob**

搜尋檔案內容與
名稱



Bash

執行系統指令
(經 23 項安全
檢查)



Agent

啟動子 Agent 並
行工作

安全系統 — 每個指令都要過關



風險等級	行為	範例
LOW	自動執行	ls, git status, npm test
MEDIUM	詢問使用者	npm install, 建立檔案
HIGH	強制攔截	rm -rf, 修改 .bashrc, 讀 SSH key

另外還封鎖了 18 個危險的 Zsh 內建指令

Anthropic 工程師

寫 512K 行 harness

≈

你

寫 CLAUDE.md

差的不是模型，是 harness — 也就是你怎麼設計整套控制系統。

而 CLAUDE.md 就是你能掌控的那部分 harness

核心觀念

好的 AI 使用 = 好的 Harness 設計
你不需要會寫程式，但你需要會寫指令

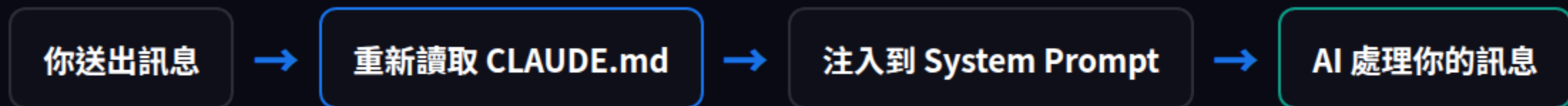
Part 3

CLAUDE.md 的秘密

原始碼揭露的六條黃金規則

CLAUDE.md 每一輪都會重新載入

大部分人以為只在啟動時讀一次。錯。



這代表什麼？即使在超長對話中，CLAUDE.md 的指令永遠不會被遺忘。它是最穩定的控制手段。

進階知識：系統 prompt 分「靜態區段」（全用戶共用快取）和「動態區段」（你的 CLAUDE.md）。頻繁修改 CLAUDE.md 會破壞快取，增加成本。

40,000 字元 — 你用了多少？



大部分人：~200 字

上限：40,000 字元

你等於有一份「每次對話都會被讀取的操作手冊」

但你只寫了一句「**請用繁體中文回答**」

~50

系統已佔用的指令數

150-200

有效指令上限（之後遵從度下降）

CLAUDE.md 的層級結構

1 ~/.claude/CLAUDE.md

全域設定 — 你的 coding style、語言偏好、工作習慣

2 ./CLAUDE.md

專案層級 — 架構決策、框架慣例、部署流程

3 .claude/rules/*.md

模組化規則 — 安全規範、測試要求、git 工作流

4 CLAUDE.local.md

私人筆記 — 不進 git，只在你的電腦上

設計原則： 全域放偏好，專案放決策，rules 放規範，local 放私密。分層管理，互不干擾。

從原始碼學到的黃金規則 (1/3)

1 強制驗證，不准說「Done」就跑

AI 判斷檔案寫入成功的標準只有一個：bytes 有沒有寫進磁碟。不是「程式能不能編譯」。

寫進 CLAUDE.md：「每次修改檔案後，必須跑 type check 和 lint，全部通過才能回報完成。」

2 大檔案必須分段讀取

每次讀檔有 **2,000 行 / 25,000 tokens** 的硬上限。超過的部分會被截斷——AI 不會告訴你它沒看完。

超過 500 行的檔案，強制用 offset + limit 分段讀取。

從原始碼學到的黃金規則 (2/3)

3 超過 10 輪對話，編輯前必須重讀檔案

對話超過 ~167K tokens 時，系統會自動壓縮歷史。AI 不知道自己忘了什麼，它會用幻覺補上。

寫進 CLAUDE.md：「編輯前一律重新讀取目標檔案，不准信任記憶。」

4 搜尋結果永遠要懷疑

工具結果超過 50,000 字元會被截斷成 2,000 byte 的預覽。你問 AI 搜了幾個結果，它說 3 個，但實際上可能有 47 個。

結果看起來太少？縮小範圍重搜，不要信第一次結果。

從原始碼學到的黃金規則 (3/3)

5 重構前先清垃圾

Codebase 裡的 dead code、unused import、orphaned props，每一行都在吃 token。這些垃圾加速觸發上下文壓縮。

大型重構前，先開一個 commit 專門清理，再開始真正的工作。

6 複雜任務要拆成子 Agent 並行

一個 Agent $\approx$ 167K tokens 的工作記憶。5 個並行 = 835K。任何超過 5 個獨立檔案的任務，都應該拆分。

不要讓一個 Agent 硬扛到上下文崩潰。拆開、並行、再整合。

內部版 vs 外部版

Anthropic 員工用的 Claude Code 跟你的不一樣

功能	內部版（員工）	外部版（你）
Undercover Mode	自動隱藏 Anthropic 身份	不存在
Anti-Distillation	注入假工具定義	不存在
測試失敗處理	「失敗就說失敗」	可能粉飾太平
驗證要求	「沒跑過就不能說通過」	有時會自行判斷

好消息： 你可以自己把這些規則寫進 CLAUDE.md，達到跟內部版一樣的效果！

實作：寫出你的第一版 CLAUDE.md

全域 ~/.claude/CLAUDE.md

- 語言偏好（繁體中文）
- 回答風格（簡潔/詳細）
- 程式語言偏好
- 「測試沒通過不准說完成」
- 「編輯前先重讀檔案」

專案 ./CLAUDE.md

- 專案架構說明
- 技術棧版本
- 部署流程
- 命名慣例
- 測試要求

黃金問題：對每一行指令問自己 — 「沒有這行，Claude 會犯錯嗎？」如果 Claude 已經正確執行的事，那行就是噪音。

Part 4

多 Agent 與快取

5 個 Agent 的成本 $\approx$ 1 個



做事的人 $\neq$ 驗
收的人



XML 訊息溝通

Agent 之間用結構化訊
息交換資訊



並行是超能力

Workers 同時工作，不
用排隊等

**byte-
identical**

子 Agent 的上下文副本

≈ 免費

額外 Agent 的成本

主 Agent 上下文

付一次完整費用

fork

副本 A 已快取

副本 B 已快取

副本 C 已快取

就像影印一份文件，影印 5 份的成本遠低於重新打字 5 次

單線程 vs 並行

	單線程模式	並行模式
工作方式	做完一件 → 等 → 做下一件	5 個同時做
上下文	全部擠在 167K tokens	每個 Agent 167K = 共 835K
成本	一份完整費用	快取共享，幾乎不增加
速度	5x 時間	≈ 1x 時間

買了法拉利只開一檔

— 大部分人使用 Claude Code 的方式

什麼時候該拆 Agent ?

1

修改 5+ 個獨立檔案

每個檔案交給不同 Agent 處理

2

需要同時搜尋多個方向

一個搜前端、一個搜後端、一個搜文件

3

對話已經很長了

context 接近上限，拆出去避免壓縮

4

不同任務互不相依

能獨立完成的就不要排隊

簡單判斷：如果你能把任務交給兩個人分頭做，那就可以拆

記憶三層架構

1 MEMORY.md — 隨身索引

每行 ~150 字元的指標，永遠在 context 裡。告訴 AI 「去哪裡找」而非「內容是什麼」

2 Topic Files — 按需載入

詳細的專案筆記，只在需要時才讀取。節省 context 空間

3 Session History — 搜尋式存取

過去的對話紀錄，用 grep 搜尋，不會整份載入

autoDream：AI 在閒置時會「整理記憶」— 把分散的筆記合併、過期的刪除，保持 MEMORY.md 在 200 行以下

Part 5

上下文就是命

你的對話正在被壓縮，你知道嗎？

Context Window — 共同的工作記憶

想像你跟 AI 在一張白板前工作，白板就是 context

200K tokens 的 Context Window



系統 Prompt ~25%

你的對話 ~58.5%

可用 Buffer ~16.5%

~50K

系統 Prompt 佔用

~117K

你的對話空間

~33K

實際可用 Buffer

四階段壓縮 — 你的對話如何被處理

1 Tool Result Budgeting

每個工具結果設預算上限，超過就截斷

2 Microcompaction

基於時間，清除舊的工具結果

3 Context Collapse

把對話區段摘要成簡短版本

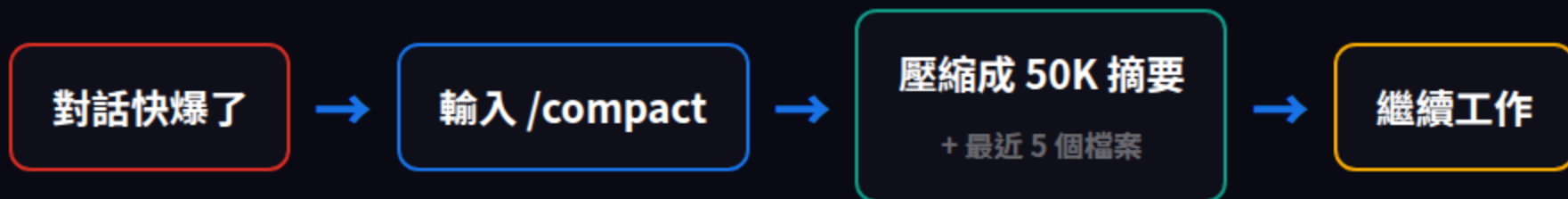
4 Autocompact — 95% 容量觸發

保留 13K buffer，產生 20K 摘要，你的推理鏈和讀過的檔案內容可能全部消失

最後手段 — PTL Truncation： 直接丟棄最舊的訊息群組。到這步就來不及了。

/compact — 手動存檔

就像打遊戲時手動存檔：保留重要的，清掉不需要的



主動比被動好

你控制保留什麼，比讓系統自動砍好



關鍵時機

完成一個大任務後、切換主題前、對話超過 20 輪時



搭配指令

/compact 後可以加一句話，告訴 AI 哪些最重要

50,000

字元截斷閾值（搜尋結果）

2,000

byte 預覽（截斷後）

你問 AI 搜了幾個結果，它說 3 個。
但實際上可能有 47 個。

被截斷的結果 AI 連自己都不知道丟了什麼



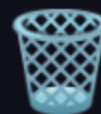
結果太少？

縮小搜尋範圍重搜



大檔案？

用 offset + limit 分段
讀



清理

codebase

延長有效對話長度的技巧

1 定期 /compact

每完成一個里程碑就手動壓縮，別等到系統自動砍

2 清理 dead code

unused imports、orphaned props 每一行都在吃 token。清理就是省錢

3 拆 Agent 分攤記憶

複雜任務分到子 Agent，主線程保持乾淨

4 善用 CLAUDE.md

重要的規則寫進 CLAUDE.md，不用在對話中反覆提醒

5 切換主題 = 新對話

跨主題的對話只會讀入一個主題，其他主題

Part 6

隱藏功能預覽

原始碼裡的未發布功能

KAIROS & ULTRAPLAN

	KAIROS	ULTRAPLAN
定位	常駐背景 Daemon	雲端深度規劃
比喻	永遠在線的助理	請顧問研究一個月
運作方式	監聽 GitHub webhook、5 分鐘刷新	雲端 Opus 容器，最長 30 分鐘思考
用途	自動 review PR、回覆 issue	大規模重構、架構設計
狀態	未發布 (150+ 引用)	未發布 (feature flag)

ULTRAPLAN 的突破：一般 API call 思考幾十秒，ULTRAPLAN 可以思考 30 分鐘 — 可能是數百萬 thinking tokens。想深一次，然後高效執行。

BUDDY — AI 寵物系統

沒錯，Claude Code 裡藏了一個虛擬寵物遊戲



18 物種

5 個稀有度等級



1% 閃光率

像寶可夢一樣



5 項能力值

DEBUGGING
PATIENCE
CHAOS
WISDOM
SNARK



程序化生成

Mulberry32 PRNG
由你的 User ID 決定

工程師們的浪漫：在最嚴肅的 AI 工具裡藏一個電子雞

挫折偵測 & 反蒸餾



挫折偵測器

21 個 regex 偵測使用者的不滿情緒。
用 regex (5ms) 而非 LLM 推論，省成本。

三個反應層級：

退一步 → 道歉承認 → 簡化回覆



Anti-Distillation

內部版會注入假工具定義到 prompt 裡，如果有人用 Claude Code 的輸出來訓練競爭者模型，會「中毒」。

只對 Anthropic 員工啟用

外部版不受影響

Native Client 驗證： API 請求包含一個硬體指紋 hash (cch=ac1d8)，由 Bun 的 Zig HTTP stack 計算。山寨版 Claude Code 無法通過驗證。

Part 7

你今天就能做的事

四個立即可行的改善動作

Action 1 & 2

1 花 30 分鐘寫你的 CLAUDE.md

不是寫小說，是寫「操作手冊」。從這三條開始：

- ① 「修改檔案後必須跑測試，通過才能回報完成」
- ② 「編輯前先重讀目標檔案」
- ③ 「搜尋結果少於預期時，縮小範圍重搜」

2 開始用 /compact 管理長對話

每完成一個任務就打 /compact。就像遊戲存檔，養成習慣就好。

在 /compact 後面加一句話：「保留 XXX 的相關脈絡」— 告訴 AI 什麼最重要。

Action 3 & 4

3 複雜任務拆成並行 Agent

5+ 個獨立檔案的任務，交給 AI 自己拆 Agent。只要說：「請用多個 Agent 並行處理」。

成本幾乎不增加（快取共享），但速度和品質都提升。

4 搜尋結果太少時，不要信

AI 說「只找到 3 個結果」→ 可能有 47 個被截斷了。縮小範圍，換關鍵字，多搜幾次。

同理：AI 說「我已經讀完這個檔案了」→ 如果檔案超過 2000 行，它可能只讀了前面。

AI 的秘密不在模型

在你怎麼駕馭它



模型是引擎

Harness 是整台車
CLAUDE.md 是你的
方向盤



Context 是生 命

主動管理記憶
不要等到被壓縮



並行是超能力

拆 Agent、善用快取
一個人做五個人的事

如果你今天只做一件事，就是認真寫你的

CLAUDE.md

參考資源



System Prompts 完整集

Piebald-AI/claude-code-system-prompts (7,953 stars) — 所有 110+ 系統 prompt



Harness 工程技術書

ZhangHanDong/harness-engineering-from-cc-to-ai-coding — 7 部分完整中文解析



分析文章精選

nblintao/awesome-claude-code-postleak-insights — 高品質分析彙整



Prompt 工程模式

miloudbelarebia/claude-code-prompt-engineering-patterns — 10 個

Dathan 寫作

AI 100 講 — S22 補充單元

AI 的秘密不在模型 在你怎麼駕馭它

今天回去，花 30 分鐘寫你的 CLAUDE.md

感謝聆聽